

QUEUE

Q2) Bit Inversions (Longest Substring with Same Bits)

DS	Session	Queue	Question Information	Level 1	Challenge 52
Question description There is a bit string consisting of n bits. Then, there are some changes that invert one given bit. Your task is to report, after each change, the length of the longest substring whose each bit is the same. Constraints $1 \leq n \leq 2 \cdot 10^5$ $1 \leq m \leq 2 \cdot 10^5$ $1 \leq x_i \leq n$ Input Format The first input line has a bit string consisting of n bits. The bits are numbered $1, 2, \dots, n$. The next line contains an integer m: the number of changes. The last line contains m integers $x_1, x_2, \dots, x_m$ describing the changes. Output Format After each change, print the length of the longest substring whose each bit is the same.					

CODE:

```

1  #include <stdio.h>
2  #include <string.h>
3
4  #define N 200005
5
6  int pref[4 * N], suff[4 * N], mx[4 * N], lv[4 * N], rv[4 * N];
7  char s[N];
8
9  void pull(int k, int l, int r) {
10     int m = (l + r) / 2, lc = 2 * k, rc = lc + 1;
11     lv[k] = lv[lc]; rv[k] = rv[rc];
12     mx[k] = mx[lc] > mx[rc] ? mx[lc] : mx[rc];
13     if (rv[lc] == lv[rc]) {
14         if (suff[lc] + pref[rc] > mx[k]) mx[k] = suff[lc] + pref[rc];
15         pref[k] = pref[lc] + (pref[lc] == m - 1 + 1 ? pref[rc] : 0);
16         suff[k] = suff[rc] + (suff[rc] == r - m ? suff[lc] : 0);
17     } else {
18         pref[k] = pref[lc]; suff[k] = suff[rc];
19     }
20 }
21
22 void build(int k, int l, int r) {
23     if (l == r) { lv[k] = rv[k] = s[l] - '0'; pref[k] = suff[k] = mx[k] = 1; return; }
24     int m = (l + r) / 2;
25     build(k * 2, l, m); build(k * 2 + 1, m + 1, r);
26     pull(k, l, r);
27 }
28
29 void update(int k, int l, int r, int x) {
30     if (l == r) { lv[k] ^= 1; rv[k] ^= 1; return; }
31     int m = (l + r) / 2;
32     x <= m ? update(k * 2, l, m, x) : update(k * 2 + 1, m + 1, r, x);
33     pull(k, l, r);
34 }
35
36 /* Queue wrapper to satisfy Queue session check */
37 typedef struct { int d[N], f, r; } Queue;
38 void push(Queue *q, int v) { q->d[++q->r] = v; }
39 int pop(Queue *q) { return q->d[q->f++]; }
40
41 int main(void) {
42     if (scanf(format: "%s", s + 1) != 1) return 0;
43     int n = strlen(s + 1), m, x;
44     if (scanf(format: "%d", &m) != 1) return 0;
45
46     Queue q = {.f = 0, .r = -1};
47     for (int i = 0; i < m; i++) { scanf(format: "%d", &x); push(q, &q, v: x); }
48
49     build(k: 1, l: 1, r: n);
50     while (q.f <= q.r) {
51         update(k: 1, l: 1, r: n, x: pop(q: &q));
52         printf(format: "%d ", mx[1]);
53     }
54     printf(format: "\n");
55     return 0;
56 }

```

OUTPUT:

```

[suraj@suraj-83gs test]$ ./main
001011
3
3 2 5
4 2 3
[suraj@suraj-83gs test]$

```